

Computer Architecture

Refers to the design, structure & organization of computer system

- It defines how processor, memory & I/O devices communicate & work together.
- it includes instruction set, m/m hierarchy & control mechanism

Technology trends - focusses on how components

like processor memory storage and network are improving with time

Ex Faster processor
improvement in wireless communication like 5G

- ~~AI, cloud computing~~
smaller components

Fast CPU with more cores

low power consumption

Moore's Law - proposed by Gordon Moore in 1965

• It states that the number of transistors on a microchip doubles approx every 2 years.

• This leads to exponential growth in computing power while size and price reduce

Application - More powerful processors, better graphics, Faster network, AI, cloud computing

Parallel computers - are designed to

process multiple tasks simultaneously to improve speed & efficiency

SISD - Single instruction single data

• Traditional computers

• Where 1 instruction operates on single data element at a time

SIMD - Single instruction multiple data

- Used in GPU
- Single instruction operates on multiple data elements

MISD - Multiple instruction single data

Used in Fault tolerant systems

- Multiple instructions operate on same data

MIMD - "M" " multiple data

- Used in multiple processors
- Each processor works independently on its own instruction and data

core and processor is an independent processing unit within CPU

eg dual core $\rightarrow$ can handle 2 tasks efficiently
quad " 4 tasks

PAGE No	
DATE	

Think of cores as workers in Factory
single core can do only 1 job at once

Myopic view of C.A

- it is a narrow perspective that focuses on improving the processor speed without considering factors like

System Cost

Energy consumption

Reliability, maintainability

A balanced view is required to ensure practical system design

used to

Amdahl's Law - Shows the limit of

speedup by parallelising only a part of program

Speedup — ratio of time taken to run
program on a single processor
to the time on multiple processor

$$S = \frac{T_1}{T_n}$$

← single processor (Sequential)
← N processor (Parallel)

measures how much faster a task executes
when parallel techniques are applied.

$S=1$ No improvement

$S=N$ Perfect speedup

$S < N \rightarrow$ Partial use

$$\text{Efficiency} = E = \frac{\text{Speedup}}{\text{no. of processors}}$$

Measures how well the processors are
utilised in a // system

Formula For speed up
and latency low

$$\text{SpeedUP} = \frac{1}{\underbrace{(1-P)}_{\text{Sequential Fraction}} + \underbrace{\frac{P}{N}}_{\text{something Factor}}} \rightarrow \text{// Fraction}$$

$P \rightarrow$ Fraction of program that can be Parallelized
 $N \rightarrow$ number of processor

eg if 80% can be // then $P = 0.8$

eg if 40% of is floating pt operation
and factor is 4 find speed up

Aptitude		Formula
Sequential	Parallel	$\frac{1}{1 - 0.4 + \frac{0.4}{4}} = \frac{1}{0.7}$
60%	40%	
no speed up	↓ Factor 4	
10	50	
0.6	0.1	
Earlier it was $\frac{1}{0.7}$		
now	0.7	

if we have 1 processor

Serial (S)

← time to execute serial part

Parallel (P)

← time to execute $n=1$

$$T_1 =$$

$$S + P$$

$$T_n =$$

$$\frac{S + P}{n}$$

$$S + \frac{P}{n}$$

assuming $T_1 = 1$

$$1 = S + P$$

$$S = 1 - P$$

Replacing in T_n

$$T_n = (1 - P) + \frac{P}{n}$$

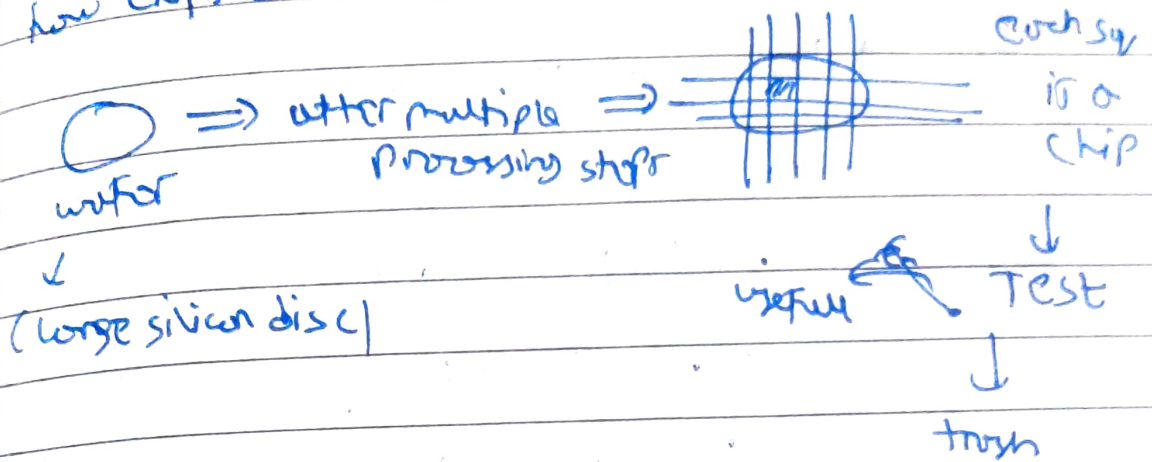
$$\text{Speed up} = \frac{T_1}{T_n} = \frac{1}{(1 - P) + \frac{P}{n}}$$

Die = chip

PAGE No.	
DATE	

Fabrication cost

how chips are made



Fabrication yield - refers to the % of Functional chips produced upon the total no of chips

$$\text{yield} = \frac{\text{Number of good chips}}{\text{Total no of chips on wafer}} \times 100$$

eg Cost of wafer = ₹2000

Number of dies per wafer = 1000

yield = 80%

• ie only 800 good die

$$\text{Cost per chip} = \frac{2000}{800} = 2.5$$

$$\text{Cost per chip} = \frac{\text{Cost of wafer}}{\text{Total no of good chip}}$$

$$\text{Yield} = e^{-ad} \rightarrow \begin{matrix} \text{die area} \\ \text{defect density} \end{matrix}$$

~~100~~

eg defect density = $0.08/\text{cm}^2$

wafer area = 150cm^2

No. of wafers per run = 500

Cost per wafer = ₹ 2000

Dies per wafer = 1000

$$\text{Yield} = e^{-\text{defect density} \times \text{area of die}} \rightarrow \frac{\text{area of wafer}}{\text{area of die}} = \frac{150}{1000}$$

$$e^{-0.08 \times 0.15}$$

$$= 0.988$$

Yield = 98.8%

Cost of production

Cost = No. of wafers $\times$ Cost per wafer

200×500

Cost per die =

No. of functional die

$= \frac{98.8}{100} \times 1000 = 988$

$$\text{Cost per die} = \frac{\text{unit cost}}{\text{no. of die}} = \frac{2000}{988} = 2.024$$

CPI → clock cycles per instruction

It is a performance metric that tells us the average number of clock cycles required to execute a single instruction in a program

Lower CPI → Faster execution

Higher CPI → slower execution

$$\text{CPI} = \frac{\text{Total clock cycles}}{\text{instruction count}}$$

← total cycles taken to execute prog

$$\text{CPU Execution Time} = \boxed{\text{C.P.I} \times \text{instruction count} \times \text{clock cycle time}}$$

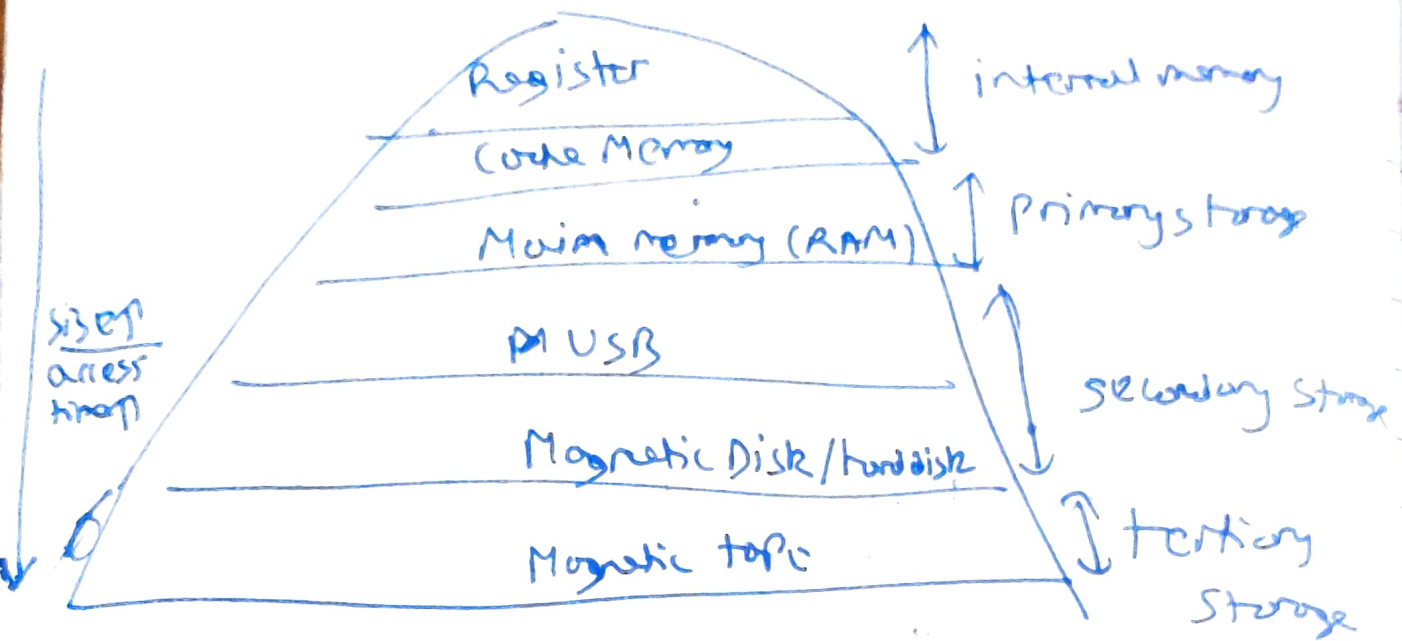
OR

$$\text{C.P.I} \times \text{instruction count}$$

$$\text{in Hz} \leftarrow \frac{\text{clock rate (freq)}}{\text{clock cycle time}}$$

$$\text{Clock cycle time} = \frac{1}{\text{clock rate}}$$

M/M hierarchy



Locality of reference

temporal locality : Recently accessed data will be used again

spatial locality : data near recently accessed data will be accessed soon

Coherence - In multiprocessor system multiple cache have copies of same data

Cache coherence ensures that all processor see the most recent value

Cache - Small, Fast memory located w/w CPU & main memory
it stores copies of frequently used data so that CPU can access

then quickly instead of going to main memory

built using SRAM (Static RAM)

works on principle of locality of reference

levels of cache L1 - closest to CPU, Fastest

L2, slightly slower

L3 - larger, slower than L1/L2 but faster than RAM

Reduce CPU access time

Mapping techniques

• Direct mapping

Fully associative memory

Set associative mapping

Unit for memory is word

Cache is divided in lines

A block from ram is placed in line in cache

basic concept

ram se data m block kaha oona h

and cache m kaha rakha and

cpu kaise access krta h

is mapping technique

When CPU want data .

1) it checks cache $\rightarrow$ if the block is there

if yes $\rightarrow$ cache hit

if no $\rightarrow$ cache miss (Fetch from RAM)

2) if block not in cache, CPU decides where in cache to place it

This is called cache mapping

Cache mapping decides how RAM blocks are placed in cache lines, so CPU knows where to look when it needs data

Direct Mapping

Each block of RAM can go to only
Fixed line in cache

• Simple, Fast

~~Disadvantage~~ 2 frequently used blocks may collide

Fully Associative mapping

- Any block of ram can go to any line
Cache line

1 word = 1 Byte

Line size = Word size

PAGE No.	
DATE	

cpu searches all lines in // to find block

Advantage - No conflict

Dis - Slower searching expensive hardware

Kary Set Associative - Cache is $\div$ into sets and each set has a few lines

A block at run is mapped to one set but inside that set it can go in any line

Adv - Better than direct and unassociated
dis - Slightly more complex

	Direct	Fully associative	Set associative
--	--------	-------------------	-----------------

Low

High

medium

Low

High

med

v. low

low

High

Slow

balanced

Fast

Lowest (due to high conflict)

Highest

medium

Hit rate

Switchability

switchable for simple systems (microcontroller)

Switchable when hit rate is critical eg GPU

Used in general purpose CPU like laptop, desktop

1 word = 1 byte

size of block = size of cache line

K way set associativity

means no. of lines in a set

eg

eg $K = 2, 4, 8$

means that divide lines in

eg cache has 16 lines

2 way set assoc.

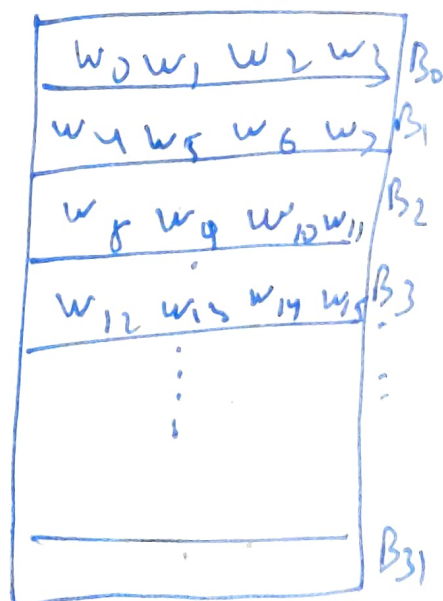
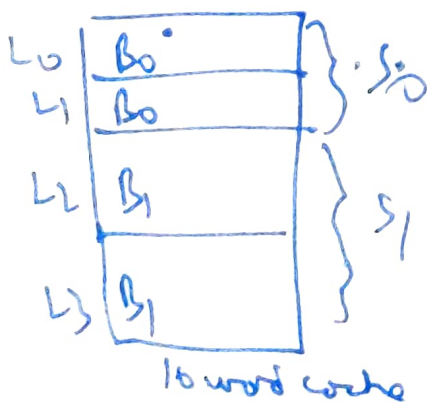
so no. of set = 8

each with 2 lines

eg

16 word cache 128 word M/M

each block can have 4 word



so no. of block = $\frac{128}{4} = 32$

lines = slots

Block size = 4 word

so line size also 4 word

so no of lines = $\frac{16}{4} = 4$

if 2 way associate ie 2 lines per set
 $k \bmod n$

no of set = $\frac{\text{Total line}}{k} = \frac{4}{2} = 2$ sets

$(k \bmod n)$] $\leftarrow$ hi block base set
 $\downarrow$ Set m jayisa
 Work

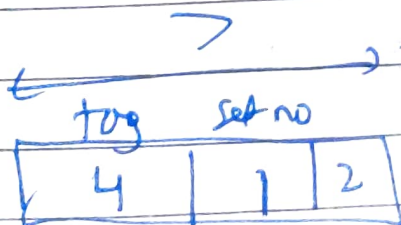
eg $0 \bmod 2$
 $1 \bmod 2$
 $2 \bmod 2$

B_0 in set 0
 B_1 in set 1
 B_2 in set 0

and set in block

hi uti ho shukh

Associativity concept



7 Address

main memory



B NO Address

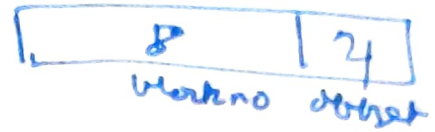
2 to represent 2 worded 1

eg tag 2
 $\downarrow$ Set 0
 0010 000
 ie work

Now in set 0 it will look for tag
 in any other lines it can get it

Other than offset rest is block no

eg offset = 4 bit
block size = 208



eg PA = 1011 0110 1100
Tag index offset

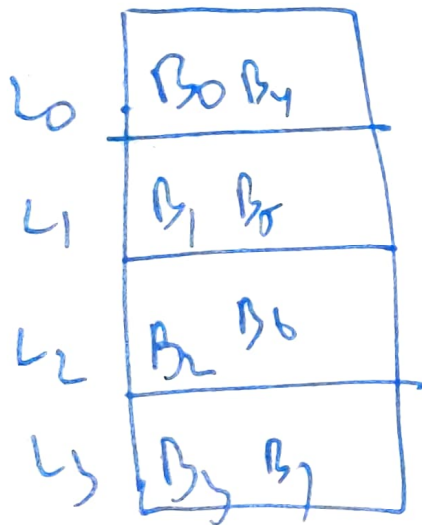
Now this is into tag and index

in line 0110 i.e 6

Check the tag 1011 if it fits there block is Tag + index
1011 0110 =

Example workstation
4 words

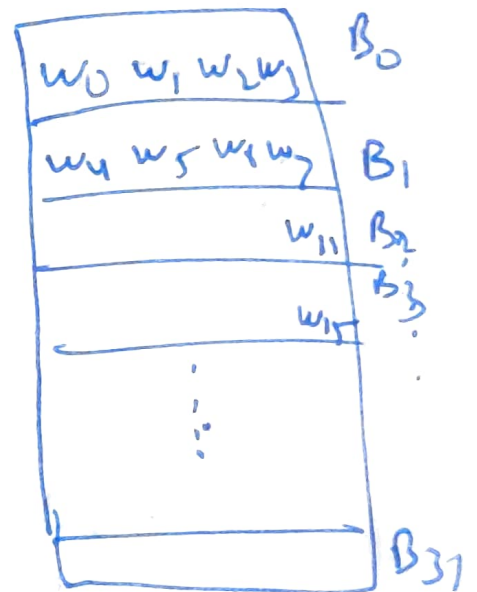
Direct mapping



Cache size 16 word

line size = block size = 4 word
so no. of line

$$\frac{\text{cache size}}{\text{line size}} = \frac{16}{4} = 4$$



128 word

For block size = 4 word

so total block
= $\frac{128}{4} = 32$

128

→ 0-127

no. of bits = $2^7 = 128$

no. of
bits
= 7

Ek line pe ek ki work m a r h e

PAGE No.	
DATE	

Direct Mapping

Formula : $K \bmod n$

$\downarrow$
worknumber

$\rightarrow$ sir nook why

Ki konsa work konsi line m jayega

work 0 $\rightarrow 0 \bmod 4 \rightarrow 0^{th}$ line

Main m/m

Physical address

how to get?

size of m/m = 128

ie 7 bits

physical

Workno	Work Offset
--------	-------------

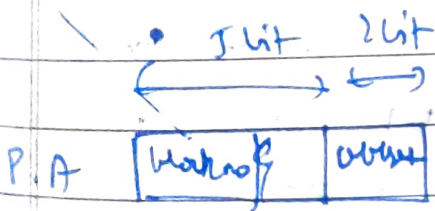
(PA) 7 bits

ek work m kitne words

ie 4

to get 4 we need

2 bit



eg if CPU says line 2

0001010

ie I need 10th word

0001010

5 2

ie 2nd work 2nd word

Why P.A. address size is not memory in cache? As

now for cache

The size remains same 7 bits



Tag
3 bit

line
no
2 bit

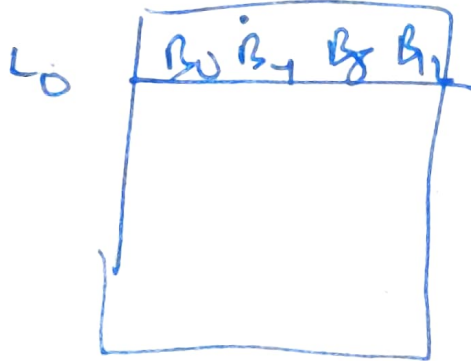
2 bit

or index
bits

4 lines so 2 bit

bit offset
remains
same as
each line
= block size

Tag tells that ki line m kanya work



Block number = Tag + line no

eg line no

Tag line no work offset

001 00 00

~~8 4 2~~
16 8 4 2

ie 8th word is in

16 work lines

See Tag & line no

Tag	line no	offset	
001	01	00	w ₁₀
001	01	01	w ₁₁
001	01	10	w ₁₂
001	01	11	w ₁₃

As a line can have only

1 block out of way
to check which
work

is present see the possible word
in line 1 right now

Cache stores copy of block with same physical address as in mem

PAGE No.	
DATE	

P.A → actual address of data in memory

In Fully associative

P.A

line ex

5	2
---	---

block no offset

Now since in cache any block can go to any line so line no is not required

P.A of cache →

5	2
---	---

Tag offset

tells us block no

i.e in a line any block can be present or 32 block so 5 will miss any block

B0 B1 ... B31
B0 B1 ... B31

w0 w1 w2 w3	B0
w4 ... w7	B1
w8 ... w11	B2

tag offset tells this

16 word

Tag offset ip

eg 00100 00 w16

 01 w17

 10 w18

 11 w19

Work 415 here

128 word

To convert decimal
in binary

First

Decimal to hex

then write it in binary

Iron law of performance - states that the

Performance of a program depends on
3 Factors :

1) no of instructions

2) Clock cycle per instruction

3) clock cycle
time

$$\text{CPU Execution time} = \text{Instruction Count} \times \text{Clock cycle time} \times \text{C.P.I.}$$

Performance $\propto$ $\frac{1}{\text{Time required for 1 clock cycle}}$

To improve the performance / make program faster

- Reduce instruction count
- Reduce C.P.I. $\rightarrow$ better architecture (pipelining)
- Reduce clock cycle time $\rightarrow$ Faster Hardware

1/12

1KB = 1024 bytes
1MB = 1024 KB

Just For understanding

CPU → Page frame
(in main memory / frame charge)

PAGE No.	
DATE	

main memory
used from data

Virtual memory - memory management technique used by modern O.S that gives the illusion of a large, continuous main memory (RAM), even if the actual physical m/m is smaller

it allows prog to believe it has access to large block of m/m by using secondary storage as an extension of m/m

- implemented using Paging or Segmentation

How virtual m/m Abstract & Extend
Physical m/m

- Abstraction means hiding the complexity and presenting a logical view

In v.m system

O.S hides the real physical m/m layout and each process sees a simple clean continuous block of m/m starting from 0 even though in reality it is scattered in RAM

This is called address space abstraction

Physical MM is limited i.e 8 GB or 16 GB
but programs may need more space than this

V.M extends physical memory by using a
portion of secondary memory or backup
MM

How it works

Only ^(currently used) active pages of prog are kept in ram

inactive pages are moved to the disk
(not currently used)

when a prog needs them they are brought into
RAM

There is a page fault and using Page Fault service
mechan they are brought in ram

Fundamental concept of V.M

- i) V.M provides address space abstraction
 - Each program is given its own virtual address

These virtual addr are mapped to physical address in RAM by the Memory management unit using a structure called page table

if any page is required that's not in MM then page fault and using page fault service mechanism it is brought into MM

Components of V.M system

1) Page table - is a data struct that is maintained by OS that is used to map virtual address to their physical address in the main memory

- keeps track of which virtual page is stored in which frame of main mem

2) P.T.E - Each entry in the page table is called page table entry

- it contains info about virtual page line

Frame number valid / invalid bit Dirty bit Reference bit

if the page is in memory or not if the page is used recently if the page is modified

Translation Lookaside Buffer - small Fast cache inside the CPU that stores recent Virtual to physical address translation

- Accessing the P.T for every translation is slow
- T.L.B reduces the overhead by storing the translation for frequently used Page table entries

CPU generates v.A

M.M.U checks T.L.B if Found, T.L.B hit Fast access
if not T.L.B miss Page table consulted

how Page table Maps

- Virtual add - Page No, offset
- M.M.U uses Page no to find the corresponding Frame Number in the Page table
- Frame number + offset gives P.A

Advantage of T, LB

- improves / reduces memory access time
- Better CPU utilization
- improved throughput
- Reduces the time required for address translation
- Stores frequently used page table entries